Author: Micheal Lanham

SECTION 1

Introducing Evaluation and Feedback

- **Purpose of evaluation:** Evaluation does not fix poor architecture by itself; it reveals actual runtime behavior and supplies a mechanism for iterative improvement tied to the Learn step in sense-plan-act-learn.
- **Feedback paths:** Results may flow back into the agent through in-loop critics and grounding checks, or into an evaluation database for regression detection, analysis, and training signals.
- **Three evaluation forms:** Deterministic tests suit exact correctness; LLM and agent evaluators handle graded language quality; human reviewers cover policy, tone, and judgment cases the automation misses.
- **Red team and benchmarks:** Red-team probes test whether specific terms or patterns break the system; benchmark suites measure goal attainment across workflows or individual agents.
- **Human feedback hygiene:** Production systems rely on user ratings and comments, but raw feedback is noisy and needs aggregation, outlier detection, stratified sampling, and review of flagged outputs.
- **Multi-agent governance:** Evaluator collusion is a real risk when judges share the same model and blind spots; mitigations include different models or personas, authority hierarchy, auditable agent-to-agent logs, escalation rules, and least-privilege authority per agent.

SECTION 2

Test-Driven Agent Development

- **TDAD mindset:** Test-driven agent development defines what good looks like as rubrics and benchmarks before building prompts and tools, mirroring TDD's discipline while accepting that teams sometimes write tests after an MVP.
- **Holistic tests:** Beyond final answers, tests can assert tool choice, reasoning steps, grounding, step count, latency, cost, and behavior when something fails.
- **Refactor ladder:** From each failure, apply the smallest change first—word, clause, sentence, section, tool, then model—to keep prompts lean and avoid contradictory multipage system prompts.
- **Stochastic reruns:** Unlike classic unit tests, LLM variability means benchmarks should run multiple times before a pass is trusted; automation can store evaluation runs offline.
- **Conflicting benchmarks:** When tests contradict each other, explicitly decide whether partial pass rates, specialist agents with routing, or a full prompt rewrite from benchmarks is the right tradeoff.
- **Agent evaluator upgrade:** Natural-language outputs need judges beyond string equality; typed pass or fail plus feedback catches formatting false negatives and scales to richer answers.

SECTION 3

Grounding, Critic, and Rubric Agents

- **Grounding agent:** Verifies that claims in an answer are supported by retrieved or authoritative context; on failure it can regenerate, block with a static response, or surface grounding metadata in the final payload.
- **Guardrails:** Output guardrails trip on ungrounded answers, enabling retry with feedback or safe failure paths instead of silent hallucination in RAG flows.
- **Critic agents:** Score generated content against criteria, block failing output, and enforce a retry ceiling with fallbacks such as human escalation, best-of-failed with confidence flags, or explicit errors.
- **Rubrics:** Structured criteria, scales, and level descriptions turn subjective quality into measurable dimensions; rubrics themselves must be validated against human judgment.
- **Structured vs open output:** Classification and extraction can use accuracy-style metrics; summaries and creative work need rubrics, LLM judges, or human review.
- **Image critic loop:** A generation agent and rubric-driven critic can iterate until a minimum score or attempt limit, with threshold calibration tied to false pass versus false fail costs.

SECTION 4

Phoenix for Traces, Experiments, and Annotations

- **Why Phoenix:** As systems mature, trace-level observability becomes necessary for debugging complex agent flows; Arize Phoenix adds projects, sessions, metrics, costs, experiments, and evaluators atop OpenTelemetry.
- **Connection pattern:** Replace default trace processors, set the collector endpoint, register a project, and wrap runs in named traces so agent workflows appear in the dashboard.
- **Metadata and sessions:** Attach session identifiers and metadata such as run id, environment, and model so traces can be filtered and cohorts compared in the UI.
- **Experiments:** Select spans into datasets, then run dashboard-driven experiments with evaluators over LLM responses, tool calls, or custom spans—the outer loop of TDAD on production-like data.
- **Annotations:** Structured labels on spans turn informal review into queryable signals that can seed regression sets and correlate quality with latency, cost, and token usage.
- **Production rule:** Agents should not ship without active evaluation, grounding or guardrails, critics, tracing, and monitored KPIs working together.